[Orquestación de Datos]

APACHE AIRFLOW

Julian Roman

[Contexto]

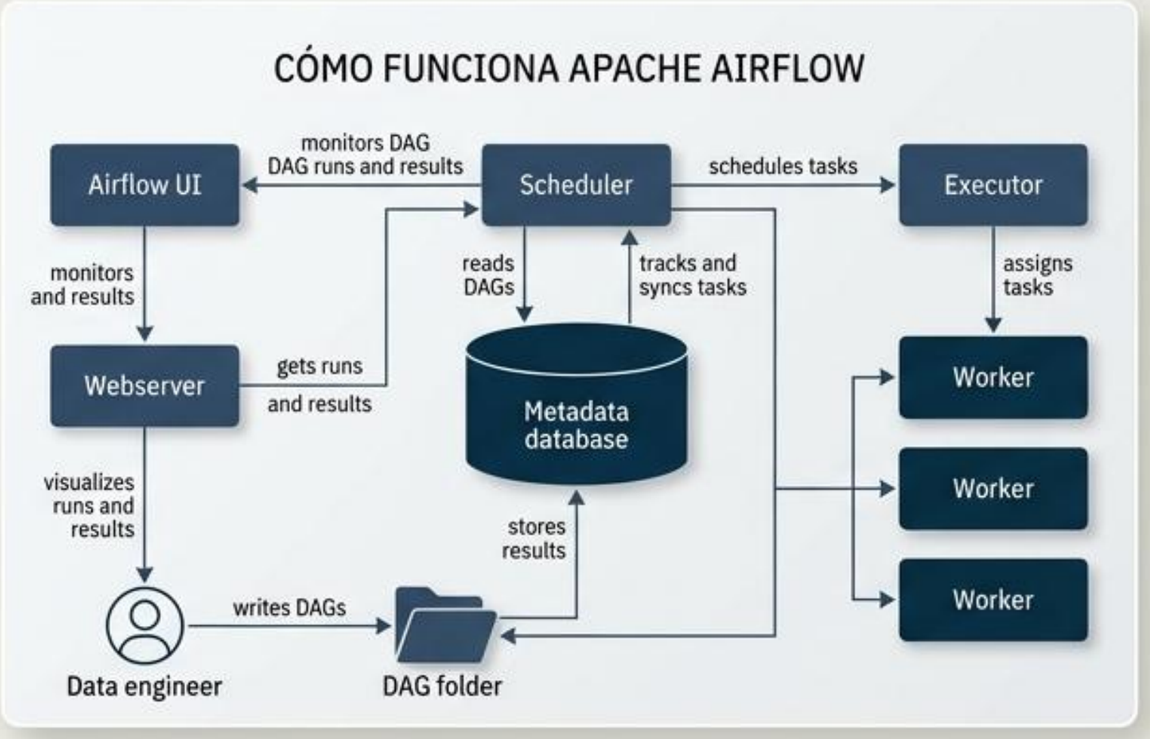
Orquestación moderna de flujos de trabajo

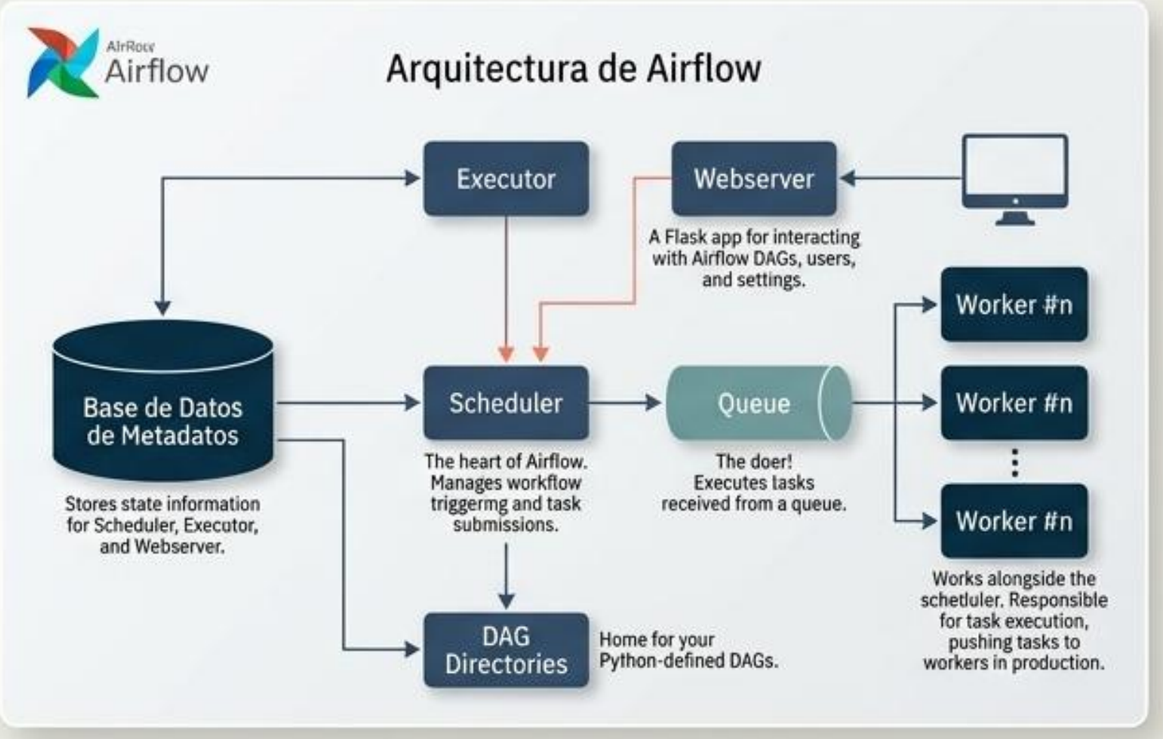
Airflow es utilizado principalmente para **orquestar flujos de trabajo** complejos en diferentes dominios, incluyendo:

- Procesamiento de datos.
- ETL (Extracción, Transformación y Carga).
- Automatización de procesos y más.

Arquitectura del Sistema Airflow

Flujo de interacción entre Scheduler, Webserver, Worker y Base de Datos.





[Arquitectura]

Anatomía de la Plataforma: Componentes Principales

DAGs

Representación del flujo de trabajo (tareas y dependencias). Definen cómo y cuándo se ejecutan, sin ciclos.

Scheduler

Monitorea DAGs y planifica tareas basándose en el schedule_interval. Asigna tareas a los Workers.

Workers

Ejecutan las tareas asignadas en paralelo (procesos múltiples o hilos) para permitir escalabilidad.

Web Server

Interfaz de usuario (UI) para visualizar DAGs, logs, ejecuciones manuales y estado general del sistema.

Metadata Database

Almacena el estado, configuración, variables, conexiones y logs (Ej. PostgreSQL, MySQL).

Executor

Configura cómo se ejecutan las tareas (Sequential, Local, Celery, Kubernetes).

[Código]

Configuración Inicial de un DAG

```python
from airflow import DAG
from airflow.operators.python_operator import PythonOperator
from airflow.operators.bash_operator import BashOperator
from airflow.operators.dummy import DummyOperator
from datetime import datetime, timedelta

# Función de ejemplo para PythonOperator
def task_1():
    print("Ejecutando la tarea 1 - PythonOperator")

# Definición de los argumentos por defecto
default_args = {
    'owner': 'airflow',
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
    'start_date': datetime(2024, 11, 15, 15, 0), # Comienza a las 3 de la tarde del 15
}

# Definición del DAG
dag = DAG(
    'three_tasks_dag',
    default_args=default_args,
    description='Un DAG con 3 tareas: PythonOperator, BashOperator y EmailOperator',
    schedule_interval='0 15 * * *', # Todos los días a las 3 de la tarde
    catchup=False, # No ejecutar tareas pasadas
)
```

```python
# Tarea 1: PythonOperator
task1 = PythonOperator(
    task_id='task_1',
    python_callable=task_1,
    dag=dag
)

# Tarea 2: BashOperator
task2 = BashOperator(
    task_id='task_2',
    bash_command='echo "Ejecutando la tarea 2 - BashOperator"',
    dag=dag
)

# Tarea 3: DummyOperator (sustituyendo el EmailOperator)
task3 = DummyOperator(
    task_id='task_3', # Simplemente una tarea de marcador de posición
    dag=dag
)

# Establecer dependencias entre las tareas
task1 >> task2 >> task3
```

[Código]

Definición de Parámetros: default_args

```python
from airflow import DAG
from airflow.operators.python_operator import PythonOperator
from airflow.operators.bash_operator import BashOperator
from airflow.operators.dummy import DummyOperator
from datetime import datetime, timedelta

# Función de ejemplo para PythonOperator
def task_1():
    print("Ejecutando la tarea 1 - PythonOperator")

# Definición de los argumentos por defecto
default_args = {
    'owner': 'airflow',
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
    'start_date': datetime(2024, 11, 15, 15, 0), # Comienza a las 3 de la tarde del 15 de nov
}

# Definición del DAG
dag = DAG(
    'three_tasks_dag',
    default_args=default_args,
    description='Un DAG con 3 tareas: PythonOperator, BashOperator y EmptyOperator',
    schedule_interval='0 15 * * *', # Todos los días a las 3 de la tarde
    catchup=False, # No ejecutar tareas pasadas
)
```

Los default_args son parámetros predeterminados aplicables a todas las tareas dentro del DAG, a menos que se sobrescriban en una tarea específica:

owner:
El propietario de la tarea (ej. 'airflow').

retries:
Número de veces que Airflow intentará ejecutar la tarea si falla (ej. 1).

retry_delay:
Tiempo de espera antes de un nuevo intento (ej. 5 minutos).

start_date:
Fecha y hora en que el DAG debe comenzar a ejecutarse (ej. 15 de noviembre de 2024, 3:00 PM).

[Código]

Programación de Ejecución y CRON

```python
# Definición del DAG
dag = DAG(
    'three_tasks_dag',
    default_args=default_args,
    description='Un DAG con 3 tareas: PythonOperator, BashOperator y EmailOperator',
    schedule_interval='0 15 * * *',  # Todos los días a las 3 de la tarde
    catchup=False,  # No ejecutar tareas pasadas
)
```

```
# * * * * *  command to execute
#
#  day of week (0 - 7)
#  month (1 - 12)
#  day of month (1 - 31)
#  hour (0 - 23)
#  min (0 - 59)
```

CRON job

¿Cómo se establece la programación?

- **schedule_interval**: Define cuándo debe ejecutarse el DAG. En este caso: '0 15 * * *' (0 minutos, 15 horas = 3:00 PM de cada día).
- **catchup=False**: Evita ejecutar tareas retroactivas de fechas pasadas. Ideal para evitar ejecuciones innecesarias si el DAG estuvo inactivo o faltaron recursos.

[Código]

Python Operator

```python
# Función de ejemplo para PythonOperator
def task_1():
    print("Ejecutando la tarea 1 - PythonOperator")

# Tarea 1: PythonOperator
task1 = PythonOperator(
    task_id='task_1',
    python_callable=task_1,
    dag=dag
)

# --- SEGUNDA COLUMNA UNIFICADA ---

import pymysql
from airflow.operators.python_operator import PythonOperator

def connect_to_mysql():
    connection = pymysql.connect(
        host='localhost',
        user='your_username',
        password='your_password',
        database='my_database',
        port=3306
    )
    if connection.open:
        print("Conexión exitosa a la base de datos.")
    elseprint("Error al conectar a la base de datos.")
    return connection
```

```python
def query_mysql(connection):
    # Crear un cursor y ejecutar la consulta
    cursor = connection.cursor()
    cursor.execute("SELECT * FROM my_table LIMIT 10;")
    results = cursor.fetchall()
    for row in results:
        print(row)
    cursor.close()
    connection.close()


connect_task = PythonOperator(
    task_id='connect_to_mysql',
    python_callable=connect_to_mysql,
    dag=dag
)

query_task = PythonOperator(
    task_id='query_mysql_task',
    python_callable=query_mysql,
    dag=dag
)

connect_task >> query_task
```

[Código]

Bash Operator

```
# Tarea 2: BashOperator
task2 = BashOperator(
    task_id='task_2',
    bash_command='echo "Ejecutando la tarea 2 - BashOperator"
    dag=dag
)
```

[Código]

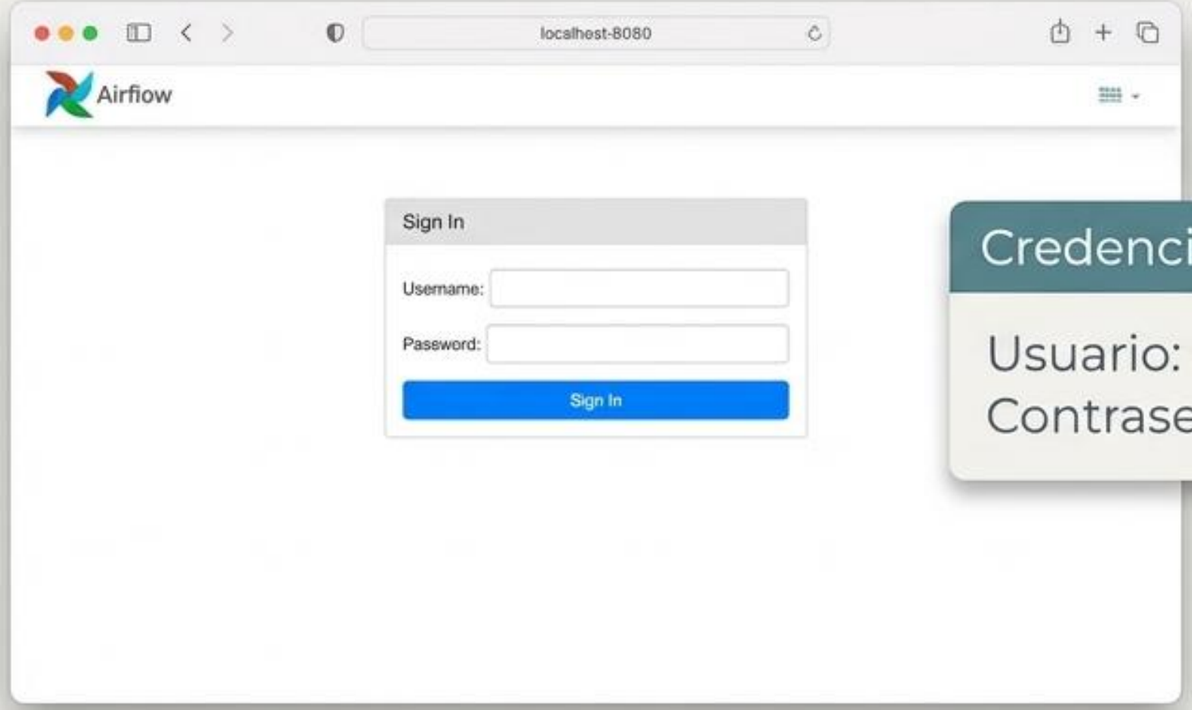
Dummy Operator

```
# Tarea 3: DummyOperator
task3 = DummyOperator(
    task_id='task_3',
    dag=dag
)
```

[Interfaz]

Despliegue e Ingreso al Entorno

```
docker-compose up -d
```

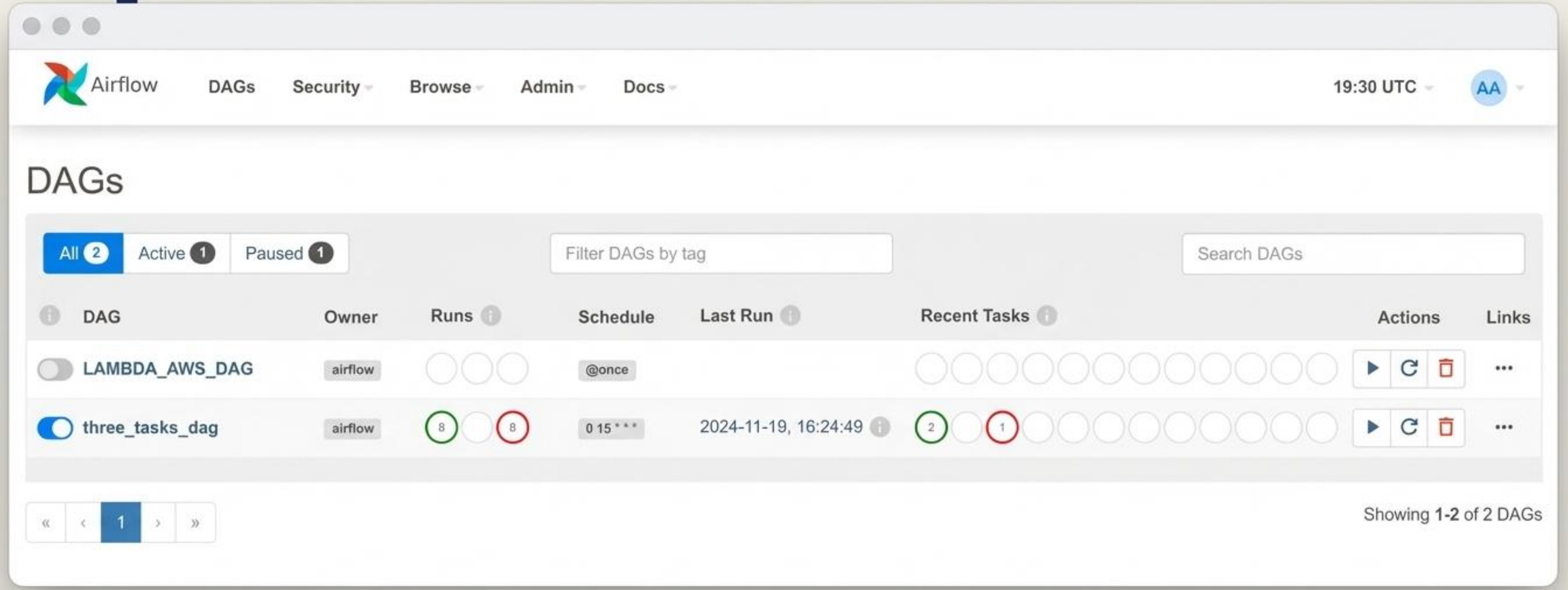


Credenciales por defecto:

Usuario: **airflow**
Contraseña: **airflow**

[Interfaz]

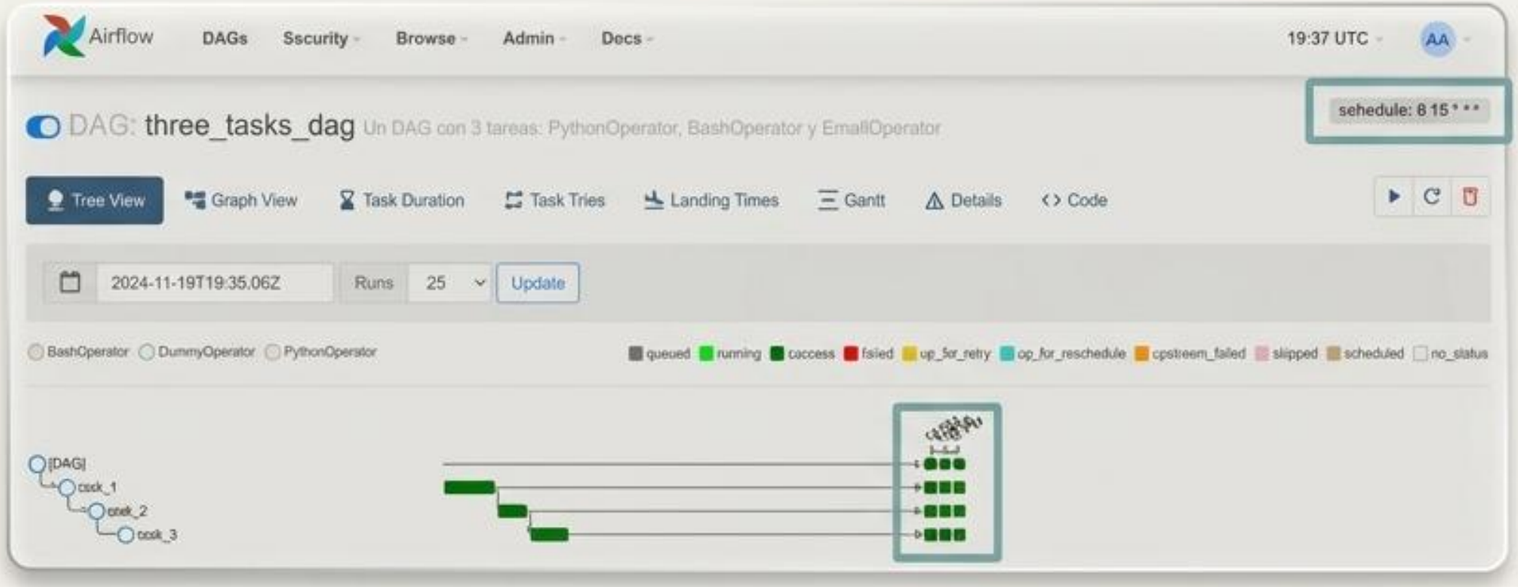
Panel de Control: DAGs Scheduler



Julian Roman

[Interfaz]

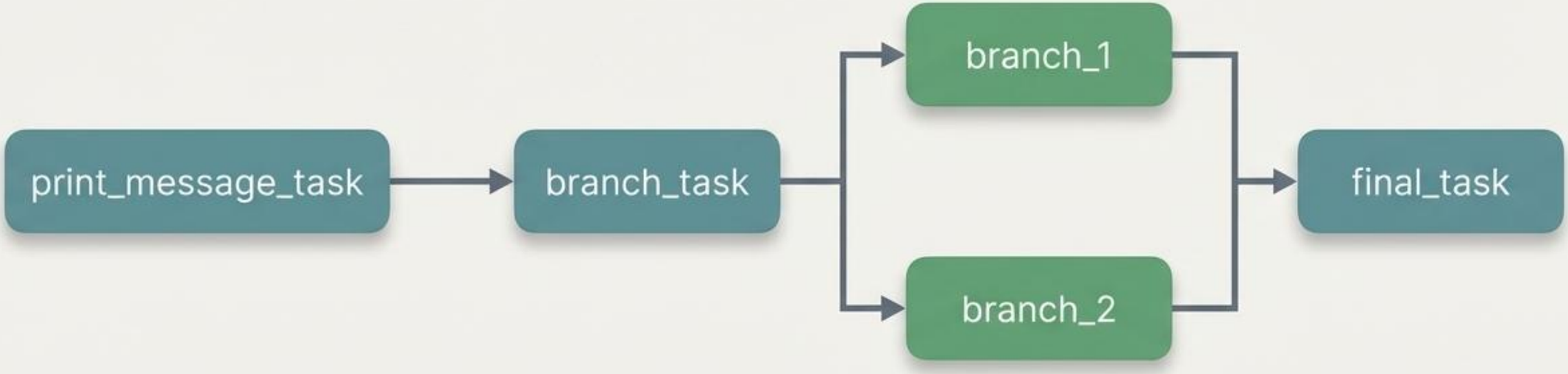
Monitoreo de Ejecución: Vistas Tree y Graph



Julian Roman

[Concepto]

Ejecución de Tareas en Paralelo



[Concepto Avanzado]

XCom: Comunicación Cruzada (Cross-Communication)

Contexto: Permite compartir información entre tareas dentro de un DAG. Una tarea comunica su resultado a otras.

Intercambio de datos:
Pasa datos de una tarea a otra en un DAG.

Almacenamiento temporal:
Los datos se guardan en la base de datos de Airflow y se recuperan mediante claves.

push
pull

Operaciones principales:
push (empujar/enviar) y pull (jalar/recuperar).

```
xcom_push: Enviar resultados de una tarea (ej. los
resultados de una consulta SQL).
```

```
xcom_pull: Recuperar los datos enviados por otra
tarea.
```

[Concepto Avanzado]

Gestión de Conexiones a Sistemas Externos

Definición:

Almacenan y gestionan credenciales a sistemas externos para no incluir información sensible directamente en el código del DAG.

Bloque 1: Seguridad

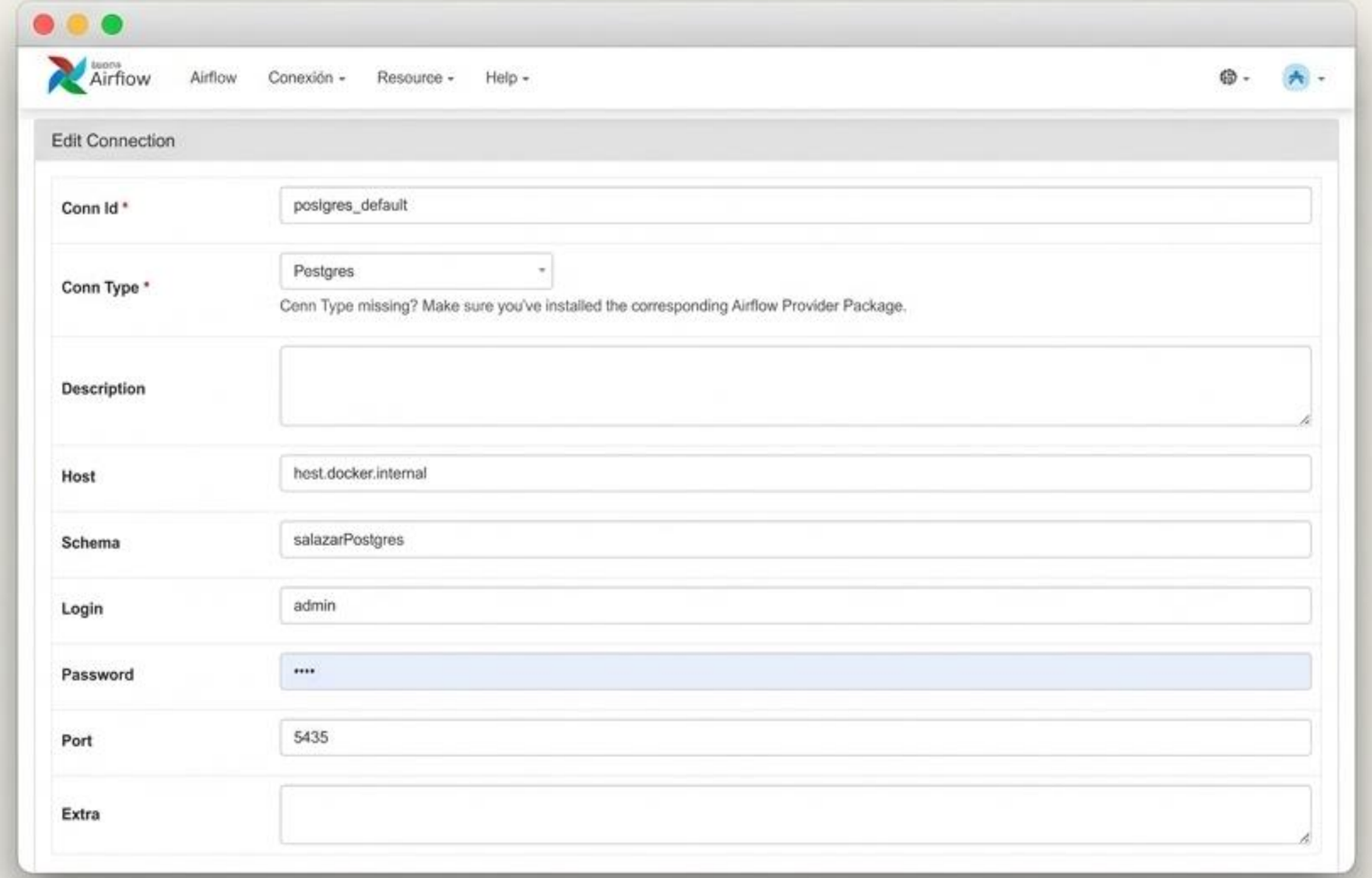
Gestión centralizada y almacenamiento seguro de credenciales.

Bloque 2: Tipos de Conexiones

- Base de datos (PostgreSQL, MySQL)
- Mensajería (Kafka, RabbitMQ)
- APIs externas (REST, SOAP)
- Nube (AWS, GCP, Azure)

Bloque 3: Uso en tareas

Los operadores se conectan a estos servicios sin tener las credenciales escritas en duro (hardcoded).

[Interfaz]

Configuración de Conexión en la UI